

语法分析

在之前的词法分析的基础之上，进一步进行语法分析。在上一步完成了对源程序进行token拆分之后，我们可以获得对应的一个token列表，接下来的语法分析就基于这个token列表进行用**递归子程序法**对文法中定义的语法成分进行分析。同时也要检测出其中的语法错误。

对于之前代码的修改

在实际写代码的过程中我发现之前设计的词法分析的程序会存在一些问题。这里首先提出来：

1. token列表的内容

首先就是之前词法分析中，一方面存在 a 类错误，也就是出现了 `&`，`|`，而我们的这个源程序是不支持的，所以这种情况会出现a类错误。当时我的做法是将这个 token 的类型定为 `TokenType.a` 类，但是实际操作中发现，对于上述的错误 token 在语法分析的过程中实际上还是把 `&` 当成 `&&`，也就是说，虽然当前出现了一个a类错误，但是在语法分析的过程中是不会管这个错误的，虽然是 `&` 但是这个token还是要当成 `TokenType.AND` 来使用，否则就会出现语法上的问题，因为在语法层面是没有这个 a 类错误对应的语法的。所以之前的 `&`，`|`，以及注释的相关代码都需要进行一定的修改。

其实修改也很简单，对于出现 `&` 和 `|` 的情况，我们都要最终把他记录成对应的正确类型的token，也就是上面说的：`&` 要记录成一个 `TokenType.AND` 的token。

```
//构建 tokenList
while ((token = lexer.nextToken()) != null) {
    if(token.getType() != TokenType.NOTE) tokenList.add(token);
}
```

2. 对于出现的错误的记录

为了记录出现的错误，我在词法分析器中增加了一个 `errorContent` 的变量，并使用 `error` 函数在发现错误的时候向其中填充错误。对于注释类型为了方便修改，最后返回的还是一个 `TokenType.NOTE` 类型的token，但是在构建整体的 `tokenList` 的时候就跳过这种类型的 token，这样就很好的解决了之前的问题。

```
// lexer 新增内容
private String errorContent = "";

private void error() {
    errorContent += line + " a\n";
}

public String getErrorContent() {
    return errorContent;
}
```

语法分析的整体构造

接下来是实现语法分析的整体代码结构。为了实现语法分析，和文法分析类似，这边重新建了一个文件夹 `syntax` 用来存放和语法分析相关的内容。在这个文件夹下除了 `Parser.java` 是用来实现语法分析的主逻辑之外，还有一个 `AST` 用来实现抽象语法树的构建。整体结构如下：

```
src
├─ Compiler.java
├─ Compiler.zip
├─ config.json
├─ lexer
│   └─ Lexer.java
│   └─ Token.java
│   └─ TokenType.java
│   └─ TokenTypeMap.java
├─ syntax
│   └─ AST
│       └─ AstNode.java
│   └─ Parser.java
├─ source
└─ SysY.g4
```

除此之外，我通过搜寻往届先辈的财产的过程中，在这个仓库 [Compiler 2023_ref Charles2530: 北航 2023年编译原理源代码](#) 中也是找到了学长说的实用插件 `antlr4`，对于整体的语法分析中存在的那么多的语法，我也是通过这个插件先对其所作的有了一个整体的了解，使得后续的代码书写变得更加的顺畅了。

这里先介绍一下这个 idea 的这个插件 `antlr4`：

我们可以创建一个 `.g4` 文件来构造自己的语法，例如我这里创建了一个文件：`SysY.g4`。文件第一行是 `grammar SysY;` 后面跟的名称必须于文件名称对应。代表我们创建的语法名称。在其中可以定义自己的语法规则，其中

1. **首字母大写**：表示 **词法规则 Lexer Rule**
2. **首字母小写**：表示 **语法规则 Parser Rule**

可以理解为如果这个首字母是大写，那他应该直接对应于 token 列表中的一些 token 类型。而如果首字母是小写就是语法结构中的一部分。很显然，在语法分析的过程中只有后面这种语法结构才能被解析，所以在这个插件中也只有语法规则可以生成对应的语法树。

在这里定义一个语法规则很简单，基本的格式就是

```
定义的名称
    : 语法规则
    ;
```

其中，和课本中的格式不太一样的就是其中的课本中用三种括号表示不同的意思：

1. `()`：课本中的 `()` 表示对于规则的提取，也就是相同部分可以提出到 `()` 外面，括号里面是剩下的内容，而在这里的 `()` 用来表示一个整体，也就是说两个东西可以放到一个 `()` 中表示一个整体，方便后面的操作。
2. `[]`：课本中的 `[]` 表示其中的内容可以是 1 次或者 0 次，而在这个地方的 `[]` 则主要是在其中表示字符集合，比如 `[0-9]` 就是说匹配任意一个数字字符，`[0-9a-z]` 就是匹配任意一个数字或者字母字符。
3. `{}`：课本中的 `{}` 表示其中的内容可以是 0 次或者多次，而在这个地方 `{}` 并没有实际的含义。

这里我们主要想要把给我们的用课本中的格式的语法转变成可以被这个 `antlr4` 解析的语法，方便我们理解，那其实就很简单，只需要按照顺序对于给我们的语法一条一条的进行解析就行了：

课本中的 `[]`，可以使用 `?`，在 `antlr` 中 `?` 表示其前面的内容可以重复 0 次或者 1 次，而 `{}` 则可以用 `*` 表示。

下面是给出的要实现的语法：

编译单元 $\text{CompUnit} \rightarrow \{\text{Decl}\} \{\text{FuncDef}\} \text{MainFuncDef}$ // 1.是否存在Decl 2.是否存在FuncDef

声明 $\text{Decl} \rightarrow \text{ConstDecl} \mid \text{VarDecl}$ // 覆盖两种声明

常量声明 $\text{ConstDecl} \rightarrow \text{'const' BType ConstDef \{'',' ConstDef\} ';'}$ // 1.花括号内重复0次 2.花括号内重复多次(花括号内重复 ≥ 2 次,即,至少出现3次ConstDef)

基本类型 $\text{BType} \rightarrow \text{'int'} \mid \text{'char'}$ // 覆盖两种基本类型

常量定义 $\text{ConstDef} \rightarrow \text{Ident} [\text{'[' ConstExp '']}] \text{'=' ConstInitVal}$ // 包含普通变量、一维数组两种情况

常量初值 $\text{ConstInitVal} \rightarrow \text{ConstExp} \mid \text{'{' [ConstExp \{'',' ConstExp\} \]'}} \mid \text{StringConst}$ // 1.常表达式初值 2.一维数组初值 3.字符串初值

变量声明 $\text{VarDecl} \rightarrow [\text{'static'}] \text{BType VarDef \{'',' VarDef\} ';'}$ // 1.花括号内重复0次 2.花括号内重复多次(花括号内重复 ≥ 2 次,即,至少出现3次VarDef) 3. static 存在 4. static 不存在

变量定义 $\text{VarDef} \rightarrow \text{Ident} [\text{'[' ConstExp '']}] \mid \text{Ident} [\text{'[' ConstExp '']}] \text{'=' InitVal}$ // 包含普通常量、一维数组定义

变量初值 $\text{InitVal} \rightarrow \text{Exp} \mid \text{'{' [Exp \{'',' Exp\} \]'}} \mid \text{StringConst}$ // 1.表达式初值 2.一维数组初值 3.字符串初值

函数定义 $\text{FuncDef} \rightarrow \text{FuncType Ident '(' [FuncFParams] ')' Block}$ // 1.无形参 2.有形参

主函数定义 $\text{MainFuncDef} \rightarrow \text{'int' 'main' '(' ')} \text{Block}$ // 存在main函数

函数类型 $\text{FuncType} \rightarrow \text{'void'} \mid \text{'int'} \mid \text{'char'}$ // 覆盖三种类型的函数

函数形参表 $\text{FuncFParams} \rightarrow \text{FuncFParam \{'',' FuncFParam\}}$ // 1.花括号内重复0次 2.花括号内重复多次(花括号内重复 ≥ 2 次,即,至少出现3次FuncFParam)

函数形参 $\text{FuncFParam} \rightarrow \text{BType Ident '[' '']}]$ // 1.普通变量 2.一维数组变量

语句块 $\text{Block} \rightarrow \text{'{' \{ BlockItem \}'}}$ // 1.花括号内重复0次 2.花括号内重复多次(花括号内重复 ≥ 2 次)

语句块项 $\text{BlockItem} \rightarrow \text{Decl} \mid \text{Stmt}$ // 覆盖两种语句块项

```

语句 Stmt → LVal '=' Exp ';' // 每种类型的语句都要覆盖
| [Exp] ';' // 有无Exp两种情况; printf函数调用
| Block
| 'if' '(' Cond ')' Stmt [ 'else' Stmt ] // 1.有else 2.无else
| 'while' '(' Cond ')' Stmt // while循环语句
| 'switch' '(' Exp ')' '{ { CaseStmt } }' // switch语句, 需覆盖不同case情况
| 'break' ';';
| 'continue' ';';
| 'return' [Exp] ';' // 1.有Exp 2.无Exp
| 'printf' '(' StringConst { ',' Exp } ')' ';' // 1.有Exp 2.无Exp

Case语句 CaseStmt → 'case' Number ':' { Stmt } | 'default' ':' { Stmt } //
1.case语句 2.default语句

表达式 Exp → AddExp // 存在即可

条件表达式 Cond → LOrExp // 存在即可

左值表达式 LVal → Ident '[' Exp ']' // 1.普通变量、常量 2.一维数组

基本表达式 PrimaryExp → '(' Exp ')' | LVal | Number // 1. 嵌套括号表达式 2. 左值
表达式 3. 数值

数值 Number → IntConst | CharConst // 存在即可, 覆盖整型常量和字符常量

一元表达式 UnaryExp → PrimaryExp | Ident '(' [FuncRParams] ')' | UnaryOp
UnaryExp | '(' BType ')' UnaryExp // 1.PrimaryExp 2.函数调用且无 FuncRParams
3.函数调用且存在 FuncRParams 4.UnaryOp UnaryExp 5.显式类型转换

单目运算符 UnaryOp → '+' | '-' | '!' 注: '!'仅出现在条件表达式中 // 三种均需覆盖

函数实参表达式 FuncRParams → Exp { ',' Exp } // 1.花括号内重复0次 2.花括号内重复多
次(花括号内重复>=2次, 即, 至少出现3次Exp) 3.Exp需要覆盖数组传参和数组元素传参

乘除模表达式 MulExp → UnaryExp | MulExp ('*' | '/' | '%') UnaryExp //
1.UnaryExp 2.* 3./ 4.% 均需覆盖

加减表达式 AddExp → MulExp | AddExp ('+' | '-') MulExp // 1.MulExp 2.+ 需覆盖
3.- 需覆盖

关系表达式 RelExp → AddExp | RelExp ('<' | '>' | '<=' | '>=') AddExp //
1.AddExp 2.< 3.> 4.<= 5.>= 均需覆盖

相等性表达式 EqExp → RelExp | EqExp ('==' | '!=') RelExp // 1.RelExp 2.==
3.!= 均需覆盖

逻辑与表达式 LAndExp → EqExp | LAndExp '&&' EqExp // 1.EqExp 2.&& 均需覆盖

逻辑或表达式 LOrExp → LAndExp | LOrExp '||' LAndExp // 1.LAndExp 2.|| 均需覆盖

常量表达式 ConstExp → AddExp 注: 使用的 Ident 必须是常量 // 存在即可

```

按照上面说的, 就可以一条一条的把这个语法规则转变成:

```

grammar SYS;

compUnit
    : decl* funcDef* mainFuncDef EOF
    ;

decl
    : constDecl
    | varDecl
    ;

constDecl
    : 'const' bType constDef (',' constDef)* ';'
    ;

```

```

bType
    : 'int'
    | 'char'
    ;

constDef
    : IDENT ('[' constExp ']')? '=' constInitVal
    ;

constInitVal
    : constExp
    | '{' (constExp (',' constExp)*)? '}'
    | STRINGCONST
    ;

varDecl
    : 'static'? bType varDef (',' varDef)* ';'
    ;

varDef
    : IDENT ('[' constExp ']')? ('=' initVal)?
    ;

initVal
    : exp
    | '{' (exp (',' exp)*)? '}'
    | STRINGCONST
    ;

funcDef
    : funcType IDENT '(' funcFParams? ')' block
    ;

mainFuncDef
    : 'int' 'main' '(' ')' block
    ;

funcType
    : 'void'
    | 'int'
    | 'char'
    ;

funcFParams
    : funcFParam (',' funcFParam)*
    ;

funcFParam
    : bType IDENT ('[' '']')?
    ;

block
    : '{' blockItem* '}'
    ;

```

```

blockItem
    : decl
    | stmt
    ;

stmt
    : lval '=' exp ';'
    | exp? ';'
    | block
    | 'if' '(' cond ')' stmt ('else' stmt)?
    | 'while' '(' cond ')' stmt
    | 'switch' '(' exp ')' '{' caseStmt* '}'
    | 'break' ';'
    | 'continue' ';'
    | 'return' exp? ';'
    | 'printf' '(' STRINGCONST (',' exp)* ')' ';'
    ;

caseStmt
    : 'case' number ':' stmt*
    | 'default' ':' stmt*
    ;

exp
    : addExp
    ;

cond
    : lorExp
    ;

lval
    : IDENT ('[' exp ']')?
    ;

primaryExp
    : '(' exp ')'
    | lval
    | number
    ;

number
    : INTCONST
    | CHARCONST
    ;

unaryExp
    : primaryExp
    | IDENT '(' funcRParams? ')'
    | unaryOp unaryExp
    | '(' bType ')' unaryExp
    ;

unaryOp
    : '+'
    | '-'

```

```

    | '!'
    ;

funcRParams
    : exp (',' exp)*
    ;

mulExp
    : unaryExp (('*' | '/' | '%') unaryExp)*
    ;

addExp
    : mulExp (('+' | '-') mulExp)*
    ;

relExp
    : addExp (('<' | '>' | '<=' | '>=') addExp)*
    ;

eqExp
    : relExp (('==' | '!=') relExp)*
    ;

lAndExp
    : eqExp ('&&' eqExp)*
    ;

lOrExp
    : lAndExp ('||' lAndExp)*
    ;

constExp
    : addExp
    ;

IDENT
    : [_a-zA-Z] [_a-zA-Z0-9]*
    ;

INTCONST
    : [0-9]+
    ;

CHARCONST
    : '\'' (ESC | ~['\\r\n]) '\''
    ;

STRINGCONST
    : '"' (ESC | ~["\\r\n])* '"'
    ;

fragment ESC
    : '\\\' [ntr0\\'"]
    ;

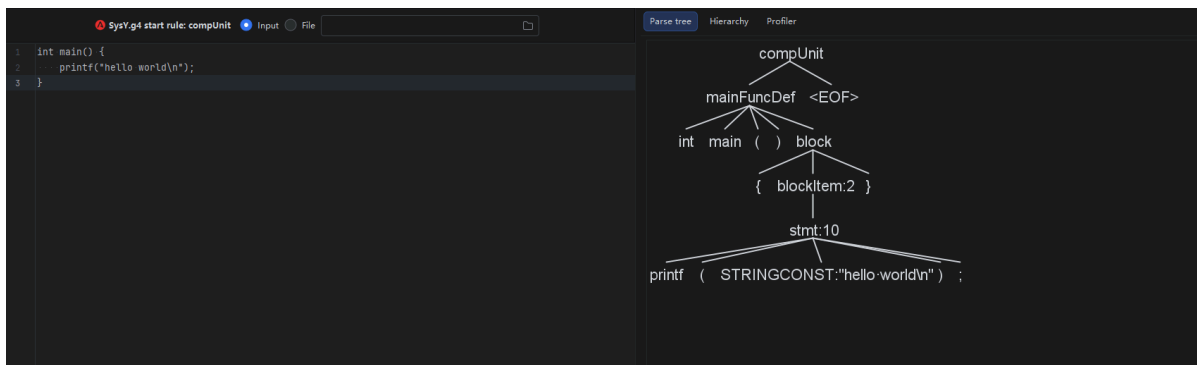
```

```
: [ \t\r\n]+ -> skip
;
```

这里其实有一点需要注意，在原来给出的文法中，表达式相关的其实都是左递归的，比如这个 `加减表达式` `AddExp → MulExp | AddExp ('+' | '-') MulExp`，我们都知道这种左递归的是会出现死循环错误的，所以在上面转变的过程中这里已经是消除了这个左递归文法了。比如这个 `addExp` 就写作：

```
addExp
: mulExp (('+' | '-') mulExp)*
;
```

写完之后我们就可以借助这个插件的力量来先对一些程序来进行解析尝尝咸淡了：比如下面是对于一个 `helloworld` 程序的解析结果：



语法分析代码

接下来进入到具体的语法分析的代码逻辑部分。其实上面完成那些对每一个语法进行解析的过程基本上就是我们接下来完成的任务，这里总结一下，这个部分主要完成了：

1. 语法结构的构建
2. 语法树的构建
3. 错误检测

语法结构的构建

和文法分析类似，这里构建了一个 `Parser` 类用来实现语法分析的主要逻辑，这里面封装了全部的语法结构。

首先，这个类包括前面词法分析结束之后得到的整个程序的 `tokenList`，那当然还需要一个指向当前 `token` 位置的指针 `pos`。其他的基本和词法分析类似。

```
private boolean isAtEnd() {
    return pos >= tokenList.size();
}

private Token peek() {
    if(isAtEnd()) return null;
    return tokenList.get(pos);
}

private Token peek(int x) {
```

```

        if(pos + x >= 0 && pos + x < tokenList.size())
            return tokenList.get(pos+x);
        return null;
    }

    private Token nextPeek() {
        if(pos+1 >= tokenList.size()) return null;
        return tokenList.get(pos+1);
    }

    private Token advance() {
        Token token;
        if(!isAtEnd()) {
            token = tokenList.get(pos++);
            if(!nodeStack.isEmpty()) {
                nodeStack.peek().addChild(token);
            }
            return token;
        }
        return null;
    }

    private TokenType peekType() {
        if(isAtEnd()) return null;
        return tokenList.get(pos).getType();
    }

    // 可以向后偏移x个单位
    private TokenType peekType(int x) {
        if(pos + x >= tokenList.size()) return null;
        return tokenList.get(pos+x).getType();
    }

    private boolean checkType(TokenType tokenType) {
        if(isAtEnd()) return false;
        return tokenType == peekType();
    }

    private void expect(TokenType tokenType) {
        if(peekType() == tokenType) {
            advance();
        } else {
            Token token = peek(-1);
            error(tokenType, token);
        }
    }
}

```

这些就不必多说了，主要就是为了防止越界以及方便我们后续对对应位置token查看的一些方法，这里要强调的是后面为了避免回溯我们会大规模的对token的类型进行检测，所以这里也写了 `checkType` 方法方便后续的操作。同时，因为在这个词法分析过程中会出现的错误只包括缺少 `;` `)` `]` 这三种，所以我这里写了一个 `expect` 方法，在可能出现上述三种符号的地方调用expect方法进行检查，如果确实缺少了，就会通过error方法记录到 `errorContent` 中。上面的方法比如说这个 `advance` 中就有了对于语法树的构建的相关内容，这个在后续内容中继续说明。

完善了上面所说的内容之后就是对于具体的语法结构的书写了。基本逻辑其实就是和之前那个 antlr 的类似，举个例子：

```
private void constDecl() {
    enterNode("<ConstDecl>");

    advance();
    bType();
    constDef();
    while(checkType(TokenType.COMMA)) {
        advance();
        constDef();
    }
    expect(TokenType.SEMICN);

    exitNode();
}
```

这里的 `enterNode` 和 `exitNode` 和 AST 的建立有关，先放一边，对于语法定义中的：`ConstDecl → 'const' BType ConstDef { ',' ConstDef } ';' ;` 如果是可以出现多次就用 `while`，如果只能出现一次就用 `if`，对于会出现的 `const` 这个 token，因为可能出现的错误不会在这个地方出现，所以直接使用 `advance` 跳过即可，最后的 `;` 使用 `expect` 进行检查，就这样就写完了一个语法结构。

这样看起来其实很简单，事实也确实如此，语法结构的构建绝大多数部分就是对于上述内容的重复，但是这里也有一些必须要解决的问题。最麻烦的就是出现了后面有多种可能的情况，比如：`InitVal → Exp | '{' [ Exp { ',' Exp } ] '}' | StringConst`，在这种情况下为了防止回溯，我们在写代码的过程中需要通过后面出现的 token 的类型来判断后续会进入那个选项中，比如上面这个，如果第一个 token 是 `{` 那就是第二种情况，如果是 `STRCON` 类型那就是最后一种，其余情况就是第一种（其实也可以通过 `Exp` 的 `first` 来判断是否进入 `Exp`，但是 `Exp` 的 `first` 比较复杂），写成代码就是：

```
private void constInitVal() {
    enterNode("<ConstInitVal>");

    if(checkType(TokenType.STRCON)) {
        advance();
    } else if(checkType(TokenType.LBRACE)) {
        advance();
        if(!checkType(TokenType.RBRACE)) {
            constExp();
            while(checkType(TokenType.COMMA)) {
                advance();
                constExp();
            }
        }
        advance();
    } else {
        constExp();
    }

    exitNode();
}
```

当然了上述的情况还是这个 first 集合比较简单情况，有的时候某个部分的 first 集合比较复杂，这个时候单独写一个判断函数会更加方便，就比如这个 decl 判断后面的内容是不是在定义变量：

```
private boolean isDeclStart() {
    if (checkType(TokenType.CONSTTK)
        || checkType(TokenType.STATICTK)) {
        return true;
    }
    if (checkType(TokenType.INTTK)
        || checkType(TokenType.CHARTK)) {
        // FuncDef: FuncType IDENT '('
        if (peekType(1) == TokenType.IDENFR
            && peekType(2) == TokenType.LPARENT) {
            return false;
        }
        // MainFuncDef
        if (peekType(1) == TokenType.MAINTK) {
            return false;
        }
        return true;
    }
    return false;
}
```

这里的逻辑其实很简单：如果当前是以 `const` 或者 `static` 开头的，那都是在进行定义，直接返回 `true`，如果是以 `int` 或者 `char` 开头的，就要急需要往后看了，这里主要要排除两种可能：定义函数，或者main函数，这两种情况返回 `false`，其余情况返回 `true`，这样就完成了对于当前是否进行 `decl` 的判断。

Stmt 语法结构

这里最困难我认为就是对于这个 stmt 的语法结构的解析了:

```

Stmt → LVal '=' Exp ';'
| [Exp] ';'
| Block
| 'if' '(' Cond ')' Stmt [ 'else' Stmt ]
| 'while' '(' Cond ')' Stmt
| 'switch' '(' Exp ')' '{' { CaseStmt } '}'
| 'break' ';'
| 'continue' ';'
| 'return' [Exp] ';'
| 'printf' '(' StringConst { ',' Exp } ')' ';'

CaseStmt → 'case' Number ':' { Stmt } | 'default' ':' { Stmt }

```

这里比较难的有两个地方：

1. `LVa1 '=' Exp ';' 还是 [Exp] ';'`

对于 stmt 进入第一种情况还是第二种情况的判断。后面的情况其实都很清晰，他们的第一个 token 类型都不一样，但是第一种 `Lval '=' Exp ';'` 和第二种 `[Exp] ';'` 比较难以区分。这里其实不难发现关键点在于这个第一种存在 `=`，而第二种不存在。但是关键在于在向后找 `=` 的过程中，你不能无限往后找，最多只能跳过一个 `lval`。写成代码其实就是：

```

        if(isAssignStmt()) {
            lval();
            advance();
            exp();
            expect(TokenType.SEMICN);
        } else {
            if(!checkType(TokenType.SEMICN)) {
                exp();
            }
            expect(TokenType.SEMICN);
        }
        break;

```

这个 `isAssignStmt()` 比较复杂。对于 `lval` 有两种：

`LVal → Ident '[' Exp '']` // 1. 普通变量、常量 2. 一维数组

注意这个判断的过程中是不能调用 `Exp()` 这个方法的，因为上面我的 `advance()` 和建立 AST 是有关的，如果在判断的时候也调用就会出现树的结构混乱的问题。

1. 很显然我先读一个Token 之后，看下一个Token 是不是 `[`，如果不是，那就是普通变量，直接判断下一个 token 是不是 `=` 就行了，如果是返回 `true`，如果不是返回 `false`，这个没得说。
2. 如果是 `[`，那就意味着这个 `lval` 是一个一维数组，因为这个 `Exp` 比较复杂，所以我们只要找到最外层的 `]` 再看看这个后面是不是 `=` 就行了。

```

private boolean isAssignStmt() {
    if (!checkType(TokenType.IDENFR)) {
        return false;
    }
    // IDENT 后面的 token
    int i = pos + 1;
    if (i >= tokenList.size()) {
        return false;
    }
    TokenType type = tokenList.get(i).getType();
    // a = 1
    if (type == TokenType.ASSIGN) {
        return true;
    }
    // a + 1 / foo() ...
    if (type != TokenType.LBRACK) {
        return false;
    }
    // 到这里说明形如 a[ ... ]
    int bracketDepth = 1;
    i++;
    while (i < tokenList.size()) {
        type = tokenList.get(i).getType();
        if (type == TokenType.SEMICN || type == TokenType.RBRACE) {
            return false;
        }
        if (type == TokenType.LBRACK) {
            bracketDepth++;
        }
    }
}

```

```

        else if (type == TokenType.RBRACK) {
            bracketDepth--;
            // 找到了最外层 a[...] 的 ]
            if (bracketDepth == 0) {
                i++;
                return i < tokenList.size()
                    && tokenList.get(i).getType()
                    == TokenType.ASSIGN;
            }
        }
        else if (type == TokenType.ASSIGN) {
            return true;
        }
        i++;
    }
    return false;
}

```

值得一提的是这样写虽然可以通过测试，但是实际上可能会出现：因为这个源程序出错导致这个 `]` 缺失的，如果正好这个 `]` 缺失，而你这样一直往后找很显然会出现错误。所以这样通过 `]` 定位 `=` 是不正确的。当然通过这个句末的 `;` 来定位这个 `=` 也不行，因为 `;` 也是可能丢失的。实际上我没有想到什么比较好的方法来在错误的还能很好的检测出来这个 `=`。但是既然上面的过了，我也就先这样了。

2. `CaseStmt` $\rightarrow$ `'case' Number ':' { Stmt } | 'default' ':' { Stmt }`

这个主要就是要判断后面是不是 `Stmt`，也就是说要判断在 `:` 后面是不是 `Stmt` 的 `first`，这样做一个 `while` 循环。我目前其实就是这样比较粗糙的判断的，虽然看起来比较唐诗，但是最后也是通过测试了，这边也就不进行进一步的优化了。

```

private boolean isStmtStart() {
    if(peekType() == TokenType.IFTK || peekType() == TokenType.WHILETK
        || peekType() == TokenType.SWITCHTK || peekType() == TokenType.BREAKTK
        || peekType() == TokenType.CONTINUETK || peekType() == TokenType.RETURNTK
        || peekType() == TokenType.PRINTFTK || peekType() == TokenType.LBRACE
        || isExpStart() || islValStart() || peekType() == TokenType.SEMICN
    ) {
        return true;
    }
    return false;
}

private boolean isExpStart() {
    if(peekType() == TokenType.IDENFR || peekType() == TokenType.LPARENT
        || peekType() == TokenType.PLUS || peekType() == TokenType.MINUS
        || peekType() == TokenType.NOT
        || peekType() == TokenType.INTCON || peekType() == TokenType.CHARCON) {
        return true;
    }
    return false;
}

private boolean islValStart() {
    if(peekType() == TokenType.IDENFR) return true;
    return false;
}

```

```
}
```

后续按照上面这样说的就能够几乎完成所有的语法结构了。

语法树的构建

在完成了上面的语法结构之后，就要考虑如何建立语法树了。这边我写了一个 `AstNode` 类用来表示这个树种的节点。

```
package syntax.AST;
import lexer.Token;
import java.io.BufferedWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

/* 抽象语法树节点类 */
public class AstNode {
    private String name;    // 当前树节点（非终结符名称）
    private Token token;    // 终结符节点才有 token
    private final List<AstNode> nodes = new ArrayList<>();

    public AstNode(String name) {
        this.name = name;
        this.token = null;
    }

    public AstNode(Token token) {
        this.name = null;
        this.token = token;
    }

    public String getName() {
        return name;
    }

    public Token getToken() {
        return token;
    }

    public void addChild(AstNode child) {
        if(child != null) {
            nodes.add(child);
        }
    }

    public void addChild(Token token) {
        AstNode node = new AstNode(token);
        nodes.add(node);
    }

    public List<AstNode> getChildren() {
        return nodes;
    }
}
```

```

public boolean isTerminal() {
    return token != null;
}

public void printTree() {
    if(!this.nodes.isEmpty()) {
        for(var node : nodes) {
            node.printTree();
        }
    }
    if(!isTerminal()) System.out.println(name);
    else System.out.println(token);
}

public void printTree(BufferedWriter bw) throws IOException {
    if(!this.nodes.isEmpty()) {
        for(var node : nodes) {
            node.printTree(bw);
        }
    }
    if(!isTerminal()) bw.write(this.name+"\n");
    else bw.write(this.token.toString()+"\n");
}
}

```

这里其实可以参考之前那个插件生成的树的结构了，可以发现只有树的叶节点才是真正的token，而树的非叶节点其实是非终结符的名称。因为每一个节点他的子节点数量是不确定，这里使用一个 List 类型来存放他的子节点。有 `addChildren` 方法加入子节点，`getChildren` 方法获取子节点列表。同时为了最后的输出，这里写了一个 `printTree` 的方法，通过对这个树进行后序遍历，这样就能输出我们想要的结果。

在设计完这个树节点的结构之后就是在我们之前写的语法结构的代码中加入相应的构建树的部分代码了。于是我在 `parser` 类中加入了几个属性：

```

private AstNode root;
private final Deque<AstNode> nodeStack = new ArrayDeque<>();

```

`root` 是构建的树的树根，`nodeStack` 是构建过程中使用的一个栈。

构建 AST 树的思路很简单，当我要对一个非终结符进行解析的时候，首先我要查看栈顶的指针，如果栈为空，那就说明这个非终结符就是树根；否则，这个非终结符就是栈顶的非终结符的子节点，我就需要把这个节点加入到栈顶的 `nodes` 中，为了递归下降的解析当前非终结符的子节点，我们需要把当前非终结符也压入栈。那当一个非终结符解析结束了，我们就需要把他弹出栈。通过这个逻辑就完成了非终结符之间的树的结构建立。

除此之外还有具体的 Token 也要放进树中，那很简单，就是通过前面的 `advance()` 方法来实现，当我们通过 `advance` 将token前进一位的时候，同时需要把当前的token对应的树节点放入栈顶的节点的子节点集合中。这样就是实现了对于AST树的建立。

具体代码如下：

```

private void enterNode(String name) {
    AstNode node = new AstNode(name);
}

```

```

    if(nodeStack.isEmpty()) {
        root = node;
    } else {
        nodeStack.peek().addChild(node);
    }

    nodeStack.push(node);
}

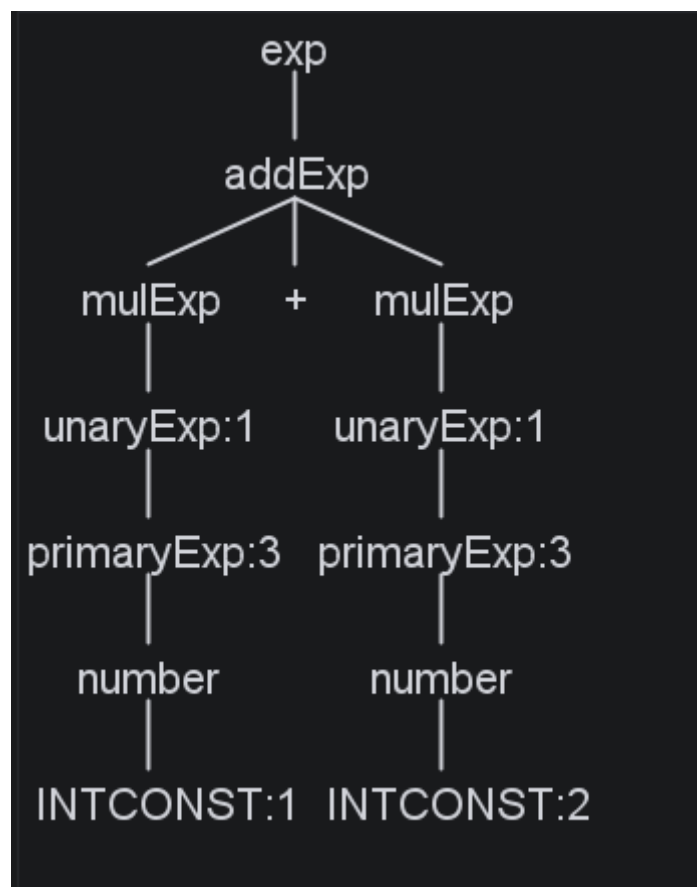
private void exitNode() {
    nodeStack.pop();
}

private Token advance() {
    Token token;
    if(!isAtEnd()) {
        token = tokenList.get(pos++);
        if(!nodeStack.isEmpty()) {
            nodeStack.peek().addChild(token);
        }
        return token;
    }
    return null;
}

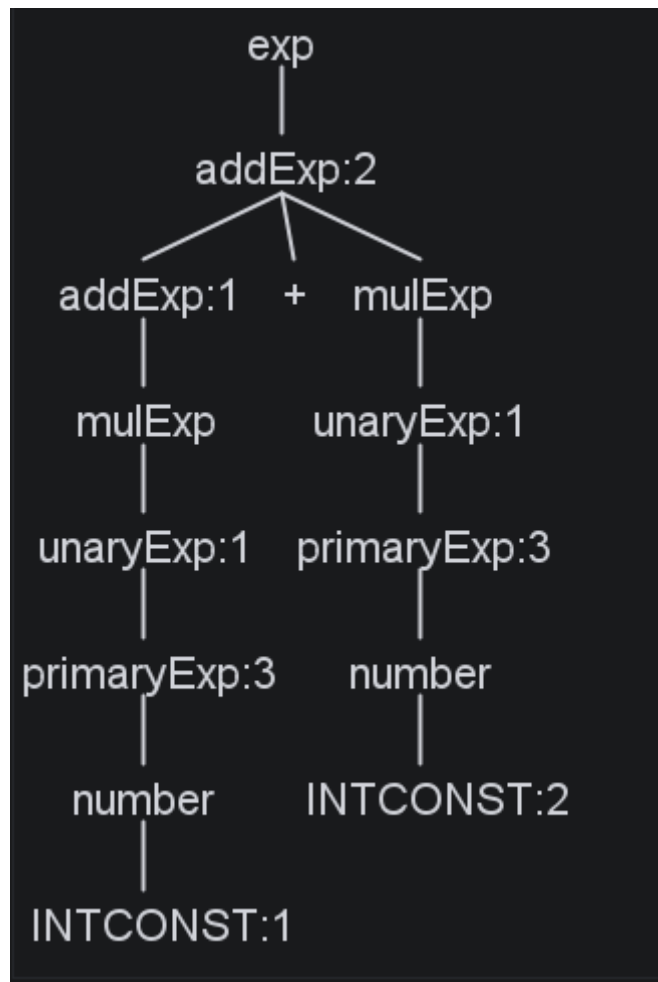
```

但是这里还有一个坑：那就是我们之前提到的，因为给我们的表达式的语法是存在左递归的，所以我们将给我们的语法去除了左递归，当前我们的代码也是按照我们修改之后的写出来的，这样就会出现一些问题：

举个例子： `1 + 2`，按照我们书写的非左递归解析出来的是这样的：



但是按照要求，我们的语法是： $\text{AddExp} \rightarrow \text{MulExp} \mid \text{AddExp} ('+' \mid '-') \text{MulExp}$ ，这样画出来的图应该是这样的：



所以我们的表达式的部分必须要有一步来对这一部分进行修改。这里使用的办法就是，对于这种表达式类型，当遇到运算符的时候，比如这里的 $1 + 2$ ，当我们处理到 2 的时候，目前 1 作为一个 mulExp 已经处理完了，并且加入到了栈顶的 addExp 的子节点中，这个时候我们应该新建一个和栈顶同名的一个节点（这里也就是 addExp），将栈顶的子节点全部放进这个新建的节点中，然后再将栈顶的子节点清空，将新建的节点加入栈顶的子节点中，后面继续这样递归处理，这样就实现了对于左递归的树的结构调整。

```
// 将已解析的内容包装成左操作数，当前节点继续接收运算符和右操作数。
private void wrapLeftOperand() {
    AstNode current = nodeStack.peek();
    AstNode left = new AstNode(current.getName());
    left.getChildren().addAll(current.getChildren());
    current.getChildren().clear();
    current.addChild(left);
}
```

那么这里 addExp 的代码也就变成了：

```

private void addExp() {
    enterNode("<AddExp>");

    mulExp();
    while(checkType(TokenType.PLUS) || checkType(TokenType.MINUS)) {
        wrapLeftOperand();
        advance();
        mulExp();
    }

    exitNode();
}

```

这样这个AST树也就建立好了。

错误处理

因为这里出现的错误只有三种：

错误类型	错误类别码	解释	对应文法
缺少分号	i	报错行号为分号 前一个非终结符 所在行号。	<code><Stmt></code> , <code><ConstDecl></code> 及 <code><VarDecl></code> 中的';'
缺少右小括号	j	报错行号为右小括号 前一个非终结符 所在行号。	函数调用(<code><UnaryExp></code>)、函数定义(<code><FuncDef></code>)及 <code><Stmt></code> 中的')'
缺少右中括号']'	k	报错行号为右中括号 前一个非终结符 所在行号。	数组定义(<code><ConstDef></code> , <code><VarDef></code> , <code><FuncFParam></code>)和使用(<code><LVal></code>)中的']'

这里我对于错误处理的思路就是使用 expect 在可能出现错误的地方进行一个检测，如果没有通过就通过 error 将这里的错误信息保存到 errorContent 中：

```

private void expect(TokenType tokenType) {
    if(peekType() == tokenType) {
        advance();
    } else {
        Token token = peek(-1);
        error(tokenType, token);
    }
}

public void error(TokenType tokenType, Token token) {
    if(tokenType == TokenType.SEMICN) {
        errorContent += token.getLine() + " i\n";
    } else if(tokenType == TokenType.RPARENT) {
        errorContent += token.getLine() + " j\n";
    } else if(tokenType == TokenType.RBRACK) {
        errorContent += token.getLine() + " k\n";
    }
}

```

这里传入 error 的 token 主要用来提供出现错误的行号。比如当前程序是：

```
a = 1
b = 2;
```

当我处理完 1 之后，当前token是这个 b ,我发现当前 token 不是 ; ，所以这个时候会记录错误，但是这个时候要注意，这里记录的错误的行号应该是这个 1 所在的位置，而不是这个 b 的位置。所以上面 expect 中的这个token是当前token的前一个token，这里要注意一下。

在完成了上面的三个部分之后，我给 Parser 弄了一个入口，因为每一个程序都是从 compUnit 开始的，所以：

```
public AstNode start() {
    compUnit();
    return root;
}
```

这样就可以对全部的语法进行分析，并输出AST树的树根了。

现在的 main 函数

在完成了上述的这些操作之后，其实还缺少的是将词法分析得到的错误和语法分析得到的错误进行一个合并的操作。因为我这个是各自读取各自的错误，最后还涉及到把每一个错误按照行号进行一个排序。这边也是动用了[一下ai神力](#)

```
List<String> errors = new ArrayList<>(error.toString().lines().toList());
errors.sort(Comparator.comparingInt(line -> Integer.parseInt(line.substring(0,
line.indexOf(' ')))));
for (String line : errors) {
    fwErr.write(line + "\n");
}
```

这边是先将 error 按行拆分并转换成 list 之后，对于其中的每一个元素，先取出他的开头到空格的子串，然后转换成数字进行比较，从小到大进行排序，最后再写回就行了。

最后的整体main是这样的：

```
import lexer.Lexer;
import lexer.Token;
import lexer.TokenType;
import syntax.AST.AstNode;
import syntax.Parser;
import utils.Tools;

import java.io.*;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class Compiler {

    public static void main(String[] args) {
```

```

    Lexer lexer;
    Token token;
    List<Token> tokenList = new ArrayList<Token>();
    StringBuilder error = new StringBuilder();

    try (BufferedReader br = new BufferedReader(new
FileReader("testfile.txt"))) {
        String source = Tools.ReadFile(br);
        lexer = new Lexer(source);
    } catch (Exception e) {
        throw new RuntimeException(e);
    }

    try (BufferedWriter fwOut = new BufferedWriter(new
Filewriter("parser.txt"));
        BufferedWriter fwErr = new BufferedWriter(new
Filewriter("error.txt"))){
        while ((token = lexer.nextToken()) != null) {
            if(token.getType() != TokenType.NOTE) tokenList.add(token);
        }
        error.append(lexer.getErrorContent());

        Parser parser = new Parser(tokenList);
        AstNode root = parser.start();
        String errorContent = parser.getErrorContent();
        error.append(errorContent);

        if(error.isEmpty()) root.printTree(fwOut);
        List<String> errors = new ArrayList<>
(error.toString().lines().toList());
        errors.sort(Comparator.comparingInt(line ->
Integer.parseInt(line.substring(0, line.indexOf(' ')))));
        for (String line : errors) {
            fwErr.write(line + "\n");
        }
    } catch (Exception e) {
        System.err.println(e);
    }
}
}

```